to represent. In the simplest cases, the 256 bit patterns might represent the integers from 0 to 255, 1 to 256, -127 to 128, etc. In a more unusual scheme, the 256 bit patterns might represent 256 exponentially related numbers: 1, 10, 100, 1000, ···, 10^{254}, 10^{255}. Everyone accessing the data must understand what value each bit pattern represents. This is usually provided by an algorithm or formula for converting between the represented value and the corresponding bit pattern, and back again.

While many encoding schemes are possible, only two general formats have become common, *fixed point* (also called integer numbers) and *floating point* (also called real numbers). In this book's BASIC programs, fixed point variables are indicated by the % symbol as the last character in the name, such as: I%, N%, SUM%, etc. All other variables are floating point, for example: X, Y, MEAN, etc. When you evaluate the formats presented in the next few pages, try to understand them in terms of their **range** (the largest and smallest numbers they can represent) and their **precision** (the size of the gaps between numbers).

Fixed Point (Integers)

Fixed point representation is used to store *integers*, the positive and negative whole numbers: ··· −3, −2, −1, 0, 1, 2, 3,···. High level programs, such as C and BASIC, usually allocate 16 bits to store each integer. In the simplest case, the $2^{16} = 65{,}536$ possible bit patterns are assigned to the numbers 0 through 65,535. This is called **unsigned integer** format, and a simplified example is shown in Fig. 4-1 (using only 4 bits per number). Conversion between the bit pattern and the number being represented is nothing more than changing between base 2 (binary) and base 10 (decimal). The disadvantage of unsigned integer is that negative numbers cannot be represented.

Offset binary is similar to unsigned integer, except the decimal values are *shifted* to allow for negative numbers. In the 4 bit example of Fig. 4-1, the decimal numbers are offset by *seven*, resulting in the 16 bit patterns corresponding to the integer numbers -7 through 8. In this same manner, a 16 bit representation would use 32,767 as an offset, resulting in a range between -32,767 and 32,768. Offset binary is not a standardized format, and you will find other offsets used, such 32,768. The most important use of offset binary is in ADC and DAC. For example, the input voltage range of -5v to 5v might be mapped to the digital numbers 0 to 4095, for a 12 bit conversion.

Sign and magnitude is another simple way of representing negative integers. The far left bit is called the **sign bit**, and is made a *zero* for positive numbers, and a *one* for negative numbers. The other bits are a standard binary representation of the absolute value of the number. This results in one wasted bit pattern, since there are two representations for zero, 0000 (positive zero) and 1000 (negative zero). This encoding scheme results in 16 bit numbers having a range of -32,767 to 32,767.